

Výstupy ŠVP**INF-08c** V programu používám podprogramy s parametry a návratovými hodnotami. (*Informatika 4, 2/2 · z INF-08*)**INF-07a** Ve vztahu k charakteru a velikosti vstupu hodnotím nároky algoritmů; porovnám algoritmy podle různých hledisek a vyberu pro řešení problém ten nejvhodnější. (*Informatika 4, 2/2 · z INF-07*)**Učivo:** programovací koncepty – podprogramy s parametry a s návratovými hodnotami**Základní pojmy****Podprogram (funkce)** – Pojmenovaná část programu, kterou lze volat opakovaně s různými vstupy: `def nazev(parametry): ... return vysledek`.**Parametr × argument** – Parametr je proměnná v hlavičce funkce (n), argument konkrétní hodnota při volání (`faktorial(5)`).**Návratová hodnota** – `return` vrátí výsledek tomu, kdo funkci zavolal, a funkci ukončí; funkce bez `return` vrací `None`.**Volání funkce** – `faktorial(5)` – výsledek lze uložit do proměnné, vypsat nebo použít ve výrazu i jako argument jiné funkce.**Lokální proměnná** – Vznikne uvnitř funkce a mimo ni neexistuje; parametry jsou také lokální.**Rekurze** – Funkce volá sama sebe s menším vstupem; musí mít základní případ, který rekurzi ukončí.**Faktoriál** – $n! = 1 \cdot 2 \cdot \dots \cdot n$ – počet pořadí n prvků; $0! = 1$.**Kombinační číslo** – $\binom{n}{k}$ – počet výběrů k prvků z n bez ohledu na pořadí.**Počet kroků** – Kolik základních operací (násobení, volání) algoritmus provede v závislosti na velikosti vstupu – měřítko jeho náročnosti.**Vzorce a vztahy** $n! = n \cdot (n-1)!$, $0! = 1$ rekurzivní definice faktoriálu
$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$
kombinační číslo přes faktoriály
$$\binom{n}{k} = \frac{n(n-1) \dots (n-k+1)}{k!}$$
součinný tvar – stačí k kroků
$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$
Pascalův trojúhelník – základ rekurzivního výpočtu**Kolik kroků výpočet potřebuje ($n = 20$, $k = 10$)**

Algoritmus	Počet kroků obecně	$n = 20$, $k = 10$
<code>faktorial(n)</code> cyklem	$n - 1$ násobení	19
<code>faktorial(n)</code> rekurzí	$n + 1$ volání	21
kombinace přes tři faktoriály	asi $2n$ násobení	37
kombinace součinným vzorcem	k násobení a k dělení	$10 + 10$
kombinace rekurzí (Pascal)	$2\binom{n}{k} - 1$ volání	369 511

Poučky a postupy

- Funkci nejdřív definuj (def), pak volej; samotná definice nic neprovede.
- Data předávej parametry a výsledek vrať `return`em – ne `input()` a `print()` uvnitř. Takovou funkci lze použít kdekoli a otestovat.
- Jedna funkce = jedna úloha; výstižný název (`faktorial`, `prumer`, `je_prvocislo`).
- Funkci otestuj samostatně (`assert faktorial(5) == 120`) – pak jí můžeš věřit ve větším programu.
- Rekurze: základní případ (`if n == 0: return 1`) a každé volání musí vstup zmenšit.
- Stejný výsledek dávají různé algoritmy s velmi různým počtem kroků – kroky počítej nebo čas měř a teprve pak vyber.

Pozor na

- `print` ve funkci není `return`: `x = funkce()` pak obsahuje `None`.
- `return` ukončí funkci; `return` uvnitř cyklu skončí hned v prvním průchodu, pokud není v podmínce.
- Rekurze bez základního případu (nebo příliš hluboká) → `RecursionError` (limit asi 1 000 úrovní).
- Proměnná vytvořená ve funkci není venku vidět (`NameError`).
- `kombinace(n, k)` pro $k > n$ nebo záporná čísla dá nesmysl – ošetři vstup.

Vzorový příkladNapiš funkce `faktorial(n)` a `kombinace(n, k)` a spočítej, kolika způsoby lze z 10 žáků vybrat trojici do soutěže.

- `def faktorial(n): vysledek = 1; for i in range(2, n + 1): vysledek = vysledek * i; return vysledek`.
- Test: `assert faktorial(0) == 1` a `assert faktorial(5) == 120`.
- `def kombinace(n, k): return faktorial(n) // (faktorial(k) * faktorial(n - k))` – celočíselné dělení, výsledek je int.
- `kombinace(10, 3) = 3628800 // (6 * 5040) = 120` trojic.
- Počet násobení: $9 + 2 + 6 = 17$; součinným vzorcem by stačily 3 násobení a 3 dělení.